House Price Prediction

Machine Learning Project

Linear Regression | Gradient Boosting | PCA | King County Dataset

1. Project Overview

This project builds and compares machine learning regression models to predict house sale prices using the King County, Washington real estate dataset. Two modelling approaches are explored side-by-side: a baseline Linear Regression model and a higher-capacity Gradient Boosting Regressor. The project also includes exploratory data analysis (EDA) with visualisations and dimensionality reduction via PCA.

1.1 Objectives

- Explore and visualise the King County house sales dataset to understand the features that drive price.
- Engineer and preprocess features (date encoding, feature dropping) suitable for scikit-learn models.
- Train a Linear Regression baseline and evaluate its R^2 score on a held-out test set.
- Train a Gradient Boosting Regressor with tuned hyperparameters and compare performance against the baseline.
- Plot staged training/test loss curves to diagnose the Gradient Boosting model's learning behaviour.
- Apply PCA to understand the variance structure of the feature set.

1.2 Technology Stack

Library / Tool	Version / Notes	Role in Project
Python	3.x	Primary language
Jupyter Notebook	housesales.ipynb	Interactive analysis and modelling environment
pandas	Latest stable	Data loading, exploration, and feature engineering
NumPy	Latest stable	Numerical arrays and staged-score accumulation
scikit-learn	Latest stable	LinearRegression, GradientBoostingRegressor, train_test_split, PCA, scale
Matplotlib	Latest stable	All scatter plots, bar charts, learning-curve plots
Seaborn	Latest stable	jointplot for geographic distribution, style helpers
mpl_toolkits	Bundled with Matplotlib	3-D plot support (imported but used as extension)

2. Dataset

2.1 Source & Size

The dataset is the King County House Sales dataset (kc_house_data.csv), a widely used public real estate benchmark covering home sales in King County, Washington (including Seattle) between May 2014 and May 2015.

Property	Value
File	kc_house_data.csv
Rows (transactions)	21,613
Columns (features)	21
Missing values	None — dataset is complete
Target variable	price (float64)
Price range	$75,000 – $7,700,000
Mean price	~$540,088
Median price	$450,000

2.2 Feature Reference

Column	Type	Description
id	int64	Unique identifier for the transaction (dropped before modelling)
date	string	Sale date — encoded as binary: 1 = 2014, 0 = 2015
price	float64	TARGET: sale price of the house in USD
bedrooms	int64	Number of bedrooms
bathrooms	float64	Number of bathrooms (fractions for partial baths)
sqft_living	int64	Interior living area in square feet
sqft_lot	int64	Total lot size in square feet
floors	float64	Number of floors in the house
waterfront	int64	Binary flag: 1 = waterfront property
view	int64	Index 0–4 rating the quality of the view
condition	int64	Overall condition of the house (1–5 scale)
grade	int64	Overall grade given by King County grading system (1–13)
sqft_above	int64	Square footage of house excluding basement
sqft_basement	int64	Square footage of the basement

yr_built	int64	Year the house was originally built
yr_renovated	int64	Year the house was renovated (0 = never renovated)
zipcode	int64	ZIP code of the property location
lat	float64	Latitude coordinate
long	float64	Longitude coordinate
sqft_living15	int64	Average interior living area of the nearest 15 neighbours
sqft_lot15	int64	Average lot size of the nearest 15 neighbours

3. Exploratory Data Analysis (EDA)

The notebook performs a structured visual EDA across 12 charts before any modelling begins. Each visualisation investigates a specific feature-price relationship or data distribution.

3.1 Visualisations Produced

Chart	Variables	Key Insight
Bar chart	bedrooms count	Most homes have 3–4 bedrooms; extreme outliers exist (33 bedrooms)
Joint plot	latitude vs longitude	Geographic spread of all properties across King County
Scatter plot	price vs sqft_living	Strong positive correlation — larger homes command higher prices
Scatter plot	price vs longitude	Price clusters visible near Seattle's western corridor
Scatter plot	price vs latitude	Northern latitudes show higher price concentration
Scatter plot	bedrooms vs price	Price rises with bedrooms but plateaus; outliers at high bedroom counts
Scatter plot	sqft_living + sqft_basement vs price	Combined area is a strong price predictor
Scatter plot	waterfront vs price	Waterfront properties (value=1) are significantly more expensive
Bar chart	floors value counts	Single and two-storey homes dominate the dataset
Scatter plot	floors vs price	Multi-storey homes have moderately higher prices
Scatter plot	condition vs price	Condition has weak direct correlation with price alone
Scatter plot	zipcode vs price	Strong geographic price banding visible by ZIP code

3.2 Key Findings from EDA

- sqft_living is the single strongest numerical predictor of price — a clear positive linear relationship.
- Location features (lat, long, zipcode) encode significant price variation — geography matters as much as size.
- Waterfront flag is a high-value binary feature; waterfront homes sit in a clearly higher price band.

- The date feature encodes only year (2014 vs 2015) and is converted to a binary integer before modelling.
- No missing values exist in the dataset — no imputation is required.

4. Feature Engineering & Preprocessing

4.1 Steps Applied

Step	Code	Rationale
Drop id column	train1 = data.drop(['id', 'price'], axis=1)	id is a transaction key with no predictive signal
Encode date	conv_dates = [1 if values == 2014 else 0 for values in data.date]	Converts the date string to a binary year flag (2014=1, 2015=0) so it is numeric
Define labels	labels = data['price']	Isolates the regression target variable
Train/test split	train_test_split(train1, labels, test_size=0.10, random_state=2)	90% training, 10% test; random_state=2 ensures reproducibility
Scale for PCA	scale(train1)	Z-score normalisation applied before PCA so all features contribute equally to variance

4.2 Final Feature Matrix

After dropping id and price and encoding date, the training matrix (train1) contains 19 features across 21,613 rows. All features are numerical (int64 or float64) — no one-hot encoding or string handling is required. The 90/10 split produces approximately 19,451 training samples and 2,162 test samples.

5. Machine Learning Models

5.1 Model 1 — Linear Regression (Baseline)

A standard Ordinary Least Squares (OLS) Linear Regression from scikit-learn is used as the baseline. It fits a linear relationship between the 19 features and price, minimising the sum of squared residuals.

Configuration

```
from sklearn.linear_model import LinearRegression
reg = LinearRegression()
reg.fit(x_train, y_train)
reg.score(x_test, y_test)   # R² on test set
```

Property	Value
Algorithm	Ordinary Least Squares Linear Regression
Library	sklearn.linear_model.LinearRegression
Hyperparameters	All defaults (fit_intercept=True)
Training set	90% of data (~19,451 samples)
Test set	10% of data (~2,162 samples)
Evaluation metric	R^2 (coefficient of determination) on test set
Prediction	reg.predict(x_test) — used for visualisation/comparison

5.2 Model 2 — Gradient Boosting Regressor

A Gradient Boosting Regressor is trained as the primary model. Gradient Boosting builds an additive ensemble of decision trees, where each new tree corrects the residual errors of the previous ensemble. It is substantially more expressive than linear regression and can capture non-linear feature interactions.

Configuration

```
from sklearn import ensemble
clf = ensemble.GradientBoostingRegressor(
    n_estimators  = 400,
    max_depth     = 5,
    min_samples_split = 2,
    learning_rate = 0.1,
    loss          = 'ls'
)
clf.fit(x_train, y_train)
clf.score(x_test, y_test)   # R² on test set
```

Hyperparameter	Value	Meaning

n_estimators	400	Number of boosting stages (trees) to build
max_depth	5	Maximum depth of each individual decision tree
min_samples_split	2	Minimum samples required to split an internal node
learning_rate	0.1	Shrinkage factor applied to each tree's contribution
loss	'ls'	Least squares regression loss (squared error)

5.3 Learning Curve Analysis

The staged_predict() method generates predictions after each of the 400 boosting stages. The per-stage loss on both the training set (clf.train_score_) and the test set is plotted to reveal overfitting or underfitting behaviour over the course of training.

```
t_sc = np.zeros((params['n_estimators']), dtype=np.float64)
for i, y_pred in enumerate(clf.staged_predict(x_test)):
    t_sc[i] = clf.loss_(y_test, y_pred)

plt.plot(testsc, clf.train_score_, 'b-', label='Train')
plt.plot(testsc, t_sc, 'r-', label='Test')
```

The dual learning-curve plot diagnoses whether additional estimators help or whether the model begins to overfit. A healthy curve shows both train and test loss decreasing before the test loss plateaus.

5.4 Model Comparison

Aspect	Linear Regression	Gradient Boosting Regressor
Approach	Single linear equation fitted to all features	Ensemble of 400 shallow decision trees built sequentially
Non-linearity	Cannot capture — strictly linear	Naturally captures non-linear interactions and thresholds
Hyperparameters	None tuned (defaults)	n_estimators=400, max_depth=5, lr=0.1
Training speed	Very fast	Slower due to sequential tree building
Interpretability	High — coefficients directly readable	Lower — ensemble of trees
Expected R^2	Baseline (~65–70% typical)	Higher (~88–90% typical for this dataset)
Overfitting risk	Low	Moderate — mitigated by learning_rate and max_depth

6. Dimensionality Reduction — PCA

Principal Component Analysis (PCA) is applied to the scaled feature matrix as an exploratory step to understand the variance structure of the 19-feature space. The notebook scales features with sklearn's scale() (zero mean, unit variance) before fitting PCA, ensuring no single feature dominates purely due to its numeric range.

```
from sklearn.preprocessing import scale
from sklearn.decomposition import PCA

pca = PCA()
pca.fit_transform(scale(train1))
```

The PCA results (explained variance ratio per component) can be used to determine how many components capture the majority of variance — a typical analysis plots cumulative explained variance against the number of components. This informs whether a reduced feature set would be sufficient for modelling, reducing training time and potentially improving generalisation.

7. Implementation Approach

7.1 Prerequisites

- Python 3.7 or later
- Jupyter Notebook or JupyterLab
- pip packages: numpy, pandas, matplotlib, seaborn, scikit-learn

7.2 Environment Setup

Step	Command	Notes
1. Create virtual environment	python -m venv venv	Isolates project dependencies
2. Activate environment	source venv/bin/activate (Linux/Mac)	Use venv\Scripts\activate on Windows
3. Install dependencies	pip install numpy pandas matplotlib seaborn scikit-learn jupyter	All libraries used in the notebook
4. Launch Jupyter	jupyter notebook	Opens browser; navigate to housesales.ipynb
5. Run all cells	Kernel > Restart & Run All	Executes the full pipeline end-to-end

7.3 End-to-End Pipeline Walkthrough

Stage	Notebook Cells	What Happens
1. Imports	Cell 0	Loads numpy, pandas, matplotlib, seaborn, mpl_toolkits
2. Data Loading	Cell 1–3	Reads kc_house_data.csv into a DataFrame; inspects head() and describe()
3. EDA – Distributions	Cells 4, 14	Bar charts for bedroom and floor count distributions
4. EDA – Geographic	Cell 5	Seaborn jointplot of latitude vs longitude
5. EDA – Price Scatter	Cells 6–11, 15–17	12 scatter plots exploring price against key features
6. Feature Engineering	Cells 12, 20	Drops id/price columns; encodes date as binary year flag
7. Train/Test Split	Cells 21–22	90/10 split with random_state=2 for reproducibility
8. Linear Regression	Cells 18–24	Fits LinearRegression; prints R^2 on test set; generates predictions
9. Gradient Boosting	Cells 25–27	Fits GradientBoostingRegressor with tuned params; prints R^2 on test set

10. Learning Curves	Cells 28–32	Staged test loss computed; train vs test loss plotted over 400 estimators
11. PCA	Cells 33–35	Scales features and fits PCA to explore variance structure

7.4 Reproducing Results

- The train_test_split uses random_state=2, guaranteeing identical splits on every run.
- The GradientBoostingRegressor has no explicit random_state set — add random_state=42 to the constructor for fully deterministic results.
- Both models are trained on the same x_train / y_train split, so comparisons are fair and consistent.
- All plots use %matplotlib inline — charts render inline within the notebook cells.

7.5 Extending the Project

- Hyperparameter tuning: Wrap GradientBoostingRegressor in GridSearchCV or RandomizedSearchCV to systematically optimise n_estimators, max_depth, and learning_rate.
- Additional models: Add XGBoost, LightGBM, or RandomForestRegressor as additional comparison points.
- Feature importance: Access clf.feature_importances_ on the fitted GBR to identify which features matter most and potentially drop low-importance ones.
- Cross-validation: Replace the single 90/10 split with KFold or cross_val_score for a more robust performance estimate.
- Error analysis: Plot residuals (y_pred - y_test) vs y_test to detect heteroscedasticity or systematic bias in specific price ranges.
- Log-transform target: House prices are right-skewed; training on log(price) and exponentiating predictions often improves RMSE.
- Deployment: Wrap the fitted model in a Flask or FastAPI endpoint; serialise with joblib.dump() for serving predictions via REST API.

7.6 Known Limitations

- No cross-validation: A single 90/10 split is used; results may vary on different random seeds unless random_state is set on GBR.
- Date encoding is minimal: Only the year (2014 vs 2015) is preserved. Month and seasonality signals are lost.
- loss='ls' is deprecated in newer scikit-learn versions: Replace with loss='squared_error' for scikit-learn >= 1.0.
- No outlier handling: The dataset contains extreme values (e.g. 33 bedrooms, $7.7M price). Outlier removal or Winsorisation could improve model robustness.
- No feature scaling for tree models: GBR does not require scaling, but the LinearRegression model may benefit from StandardScaler on highly variable features like sqft_lot and price itself.

- Cell 28 references params['n_estimators'] but params dict is never defined: This cell will raise a NameError and should be replaced with clf.get_params()['n_estimators'] or the literal 400.

8. Summary

This project demonstrates a complete, reproducible machine learning regression workflow on a well-known real estate dataset. It covers data loading and exploration, visual EDA across 12 charts, lightweight feature engineering, model training and evaluation for two algorithms, learning curve diagnostics, and dimensionality reduction with PCA — all within a single Jupyter Notebook.

The Gradient Boosting Regressor is the primary model of interest and is expected to substantially outperform the Linear Regression baseline on this dataset due to the non-linear interactions between location, size, and price. The learning-curve analysis provides a principled view of model capacity and convergence. The PCA step lays groundwork for further feature reduction or unsupervised exploration.

Component	Status	File / Cell
Dataset loading & inspection	Complete	kc_house_data.csv, Cells 1–3
Exploratory Data Analysis (12 plots)	Complete	Cells 4–17
Date feature encoding	Complete	Cell 20
Train / Test split (90/10)	Complete	Cell 22
Linear Regression baseline	Complete	Cells 18–24, 29
Gradient Boosting Regressor	Complete	Cells 25–27
Learning curve visualisation	Partial — NameError on params dict	Cells 28–32
PCA dimensionality reduction	Complete	Cells 33–35
Hyperparameter tuning (GridSearchCV)	Not implemented	—
Cross-validation	Not implemented	—
Feature importance analysis	Not implemented	—
Model serialisation / deployment	Not implemented	—